



大家好

欢迎加入 LightDir (光向) 研习社
欢迎大家一同探索开源共享的知识分享模式

今日内容



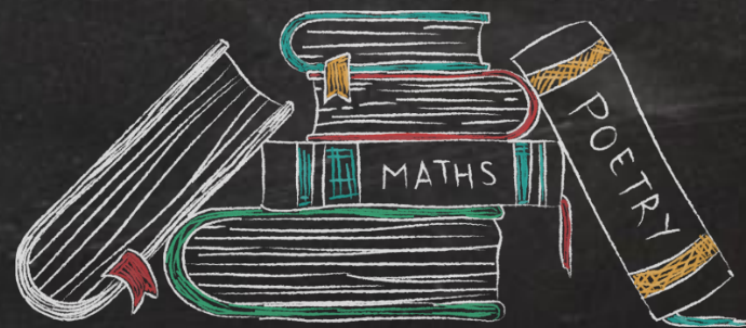
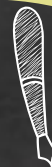
 作业·点评 答疑

 作业·示范·分析

$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

1

作业·点评 答疑



01 一组

01 一组



02 二组



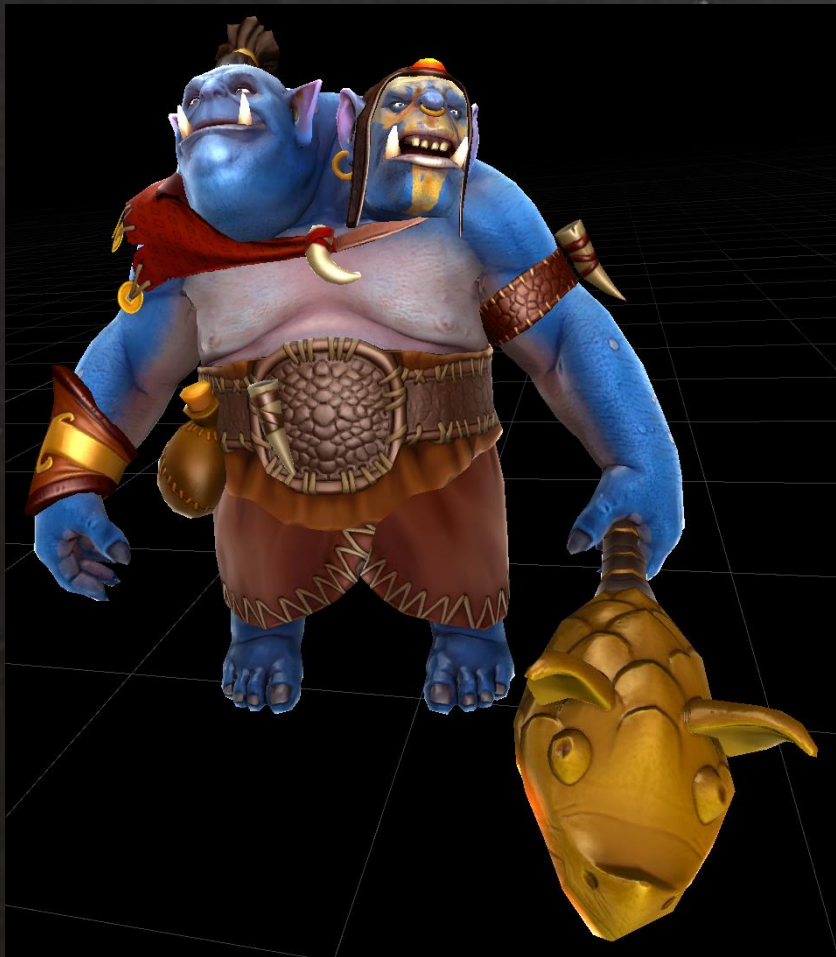
03 三组

03 三组



04 四组

04 四组



05 总结

姓名	代码作业	连连看作业	截图	打包
周川	✓	✓	✓	✓
赵井才				
叶小芸				
赵翔	✓	✓	✓	✓
张				
周翰林	✓	✓	✓	✓
杨易	✓	✓	✓	✓
廖宴楠	✓	✓	✓	✓
冯超越	✓	✓	✓	✓
邵志刚	✓	✓	✓	✓
申伏琳	✓	✓	✓	✓
陈沛霖	✓	✓	✓	✓
顾友海	✓	✓	✓	✓
冷翰林	✓	✓	✓	✓
王岩	✓	✓	✓	✓
宋歌	✓	✓	✓	✓
冯晓晨	✓	✓	✓	✓
罗陈	✓	✓	✓	✓
昊				
沈术磐	✓	✓	✓	✓

作业情况：

- 独立完成的同学都是3星，可以给自己鼓掌了；
- 可以看到用类似的技术做同样的东西，还是会有好看和更好看的区别，这是美术向TA要磨练的点；
- 对于在职的同学，建议可以找团队协商，尝试承担一些TA的工作内容；

06 答疑

- 透贴阴影问题:

- 计算投影必须要有一个ShadowCastPass, 之所以我们之前没写也对, 是因为Fallback了Diffuse, Unity会自动调用Diffuse这个Shader的ShadowCastPass, 但是它是不支持透贴的;
- 可以Fallback到一个支持AlphaCutout的Shader上, 比如: Legacy Shaders/Transparent/Cutout/VertexLit; 需要注意的是, 必须包含Shader中透明相关的所有面板属性;

- 透贴双面问题:

- 关闭剔除背面, 在声明BlendMode的地方加入: Cull Off 即可;

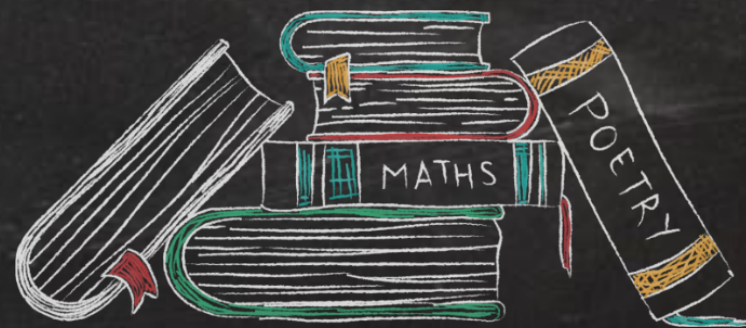
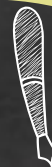
- 开线性空间做, Gamma空间看不正常的问题:

- 目前课程内容不会涉及线性空间, Gamma空间;
- 如果按公共件的要求, Shader应该支持线性和Gamma效果一致;

$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

2

作业·示范·分析



01 文档分析

▪ SOURCE 2 TEXTURES AND SHADER MASKS

- Color
- Transparency
- Normal Map
- Detail Mask
- Metalness Mask
- Self Illumination Mask
- Specular Mask
- Rimlight
- Base Tint Mask
- Specular Exponent
- Diffuse Mask (rarely used)

- Color贴图，透明贴图，法线贴图，自发光贴图，不赘述；
- DetailMask特殊效果相关，DiffuseMask罕见使用，不考虑；
- Metalness 金属度贴图，仅为金属遮罩，区别于PBR的金属度；
- Specular Mask，镜面反射遮罩，可理解为高光强度图；
- Rimlight，边缘光效果，对应有RimMask，边缘光强度图；
- Base Tint Mask，高光染色遮罩，控制高光颜色从Color贴图混合的程度；
- Specular Exponent，高光次幂，不赘述；

02 资源分析

• 非共用贴图9张:

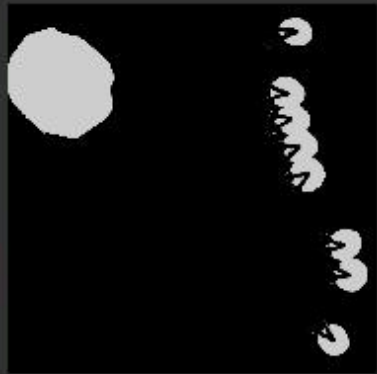
- Color (必选)
- MetalnessMask (可选)
- Normal (必选)
- RimMask (必选)
- SelfIllumMask (可选)
- SpecularExponent (必选)
- SpecularMask (必选)
- TintByBaseMask (必选)
- Translucency (可选)

• 共用贴图5张:

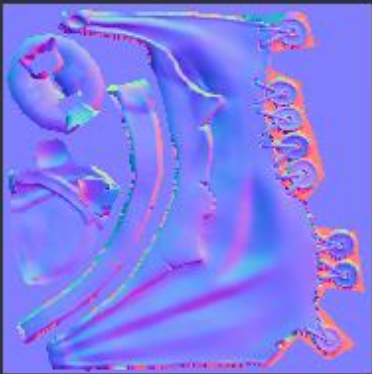
- Cubemap
- DiffuseWarp: RampTex
- FresnelWarpColor
- FresnelWarpRim
- FresnelWarpSpec



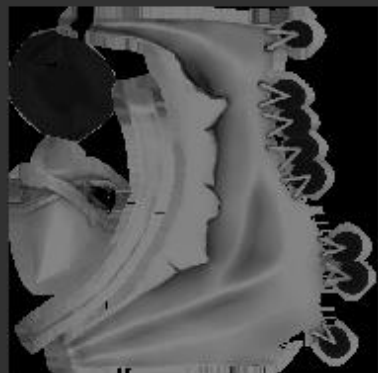
Color



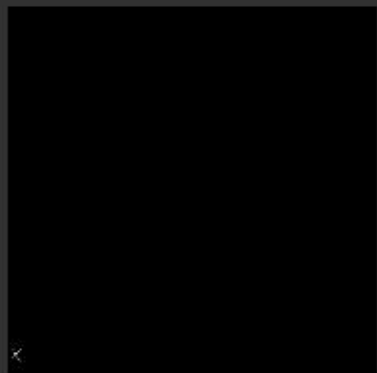
MetalnessMask



Normal



RimMask



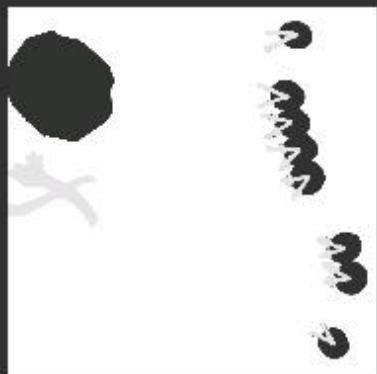
SelfIllumMask



SpecularExponent



SpecularMask



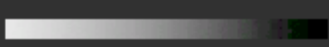
TintByBaseMask



Translucency



DiffuseWarp



FresnelWarpColor



FresnelWarpRim



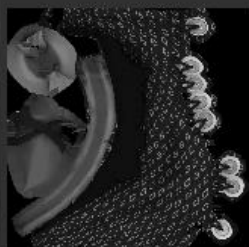
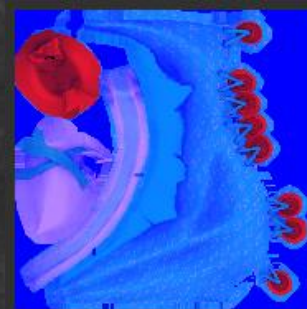
FresnelWarpSpec



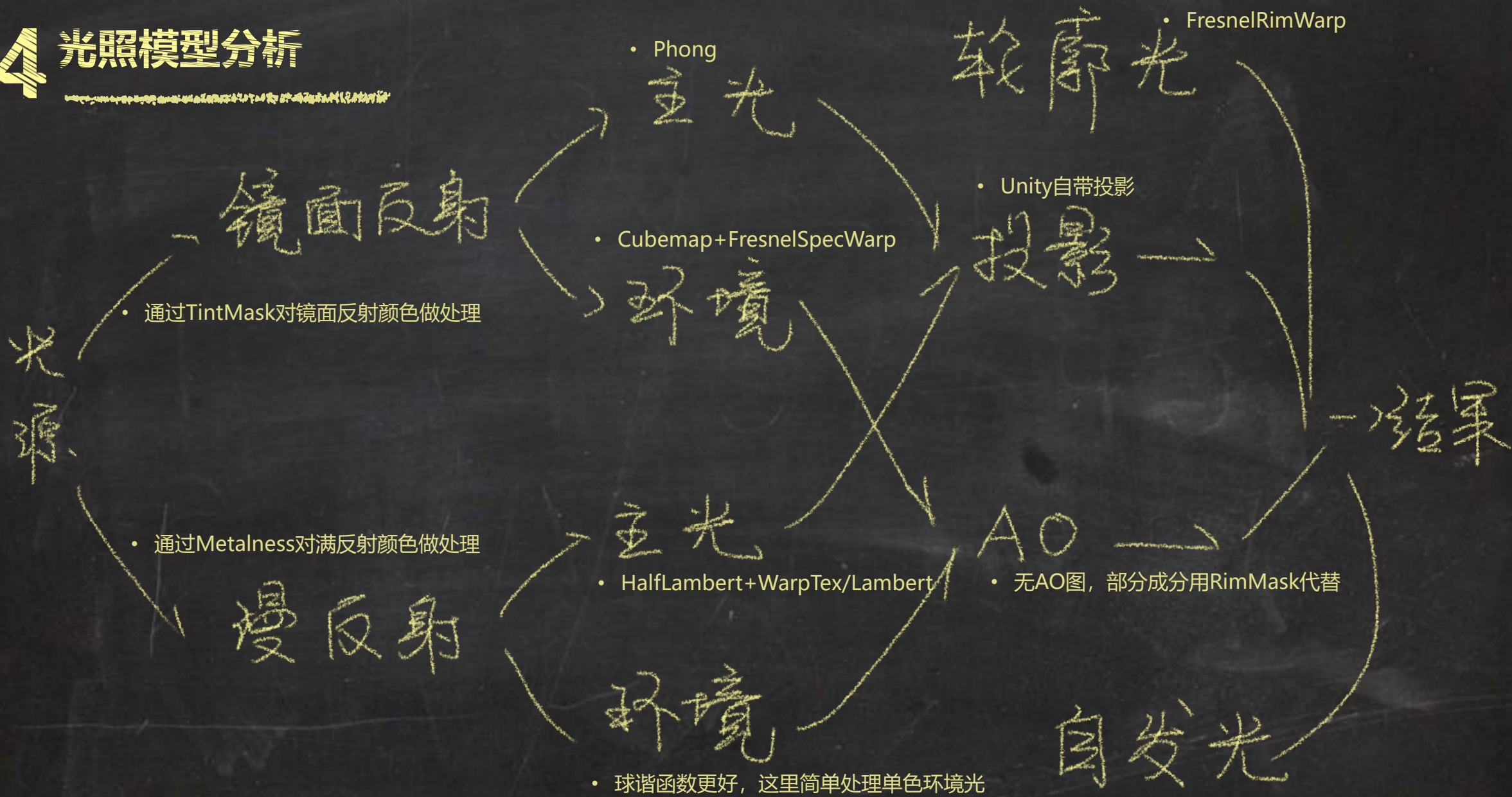
Cubemap

03 资源优化

- Color和透贴合并为一张，常规操作；
 - RGB: Color
 - A: Opacity
- SpecularMask, RimMask, TintByBaseMask, SpecularExponent, 四张为必选纹理，合并为一张，减少采样次数；
 - R: SpecInt
 - G: RimInt
 - B: TintMask
 - A: SpecPow
- FresnelWarpColor, FresnelWarpRim, FresnelWarpSpec, 三张都是对Fresnel结果的Warp，可以一次采样出来，故合并为一张；
 - R: FresnelCol
 - G: FresnelRim
 - A: FresnelSpec

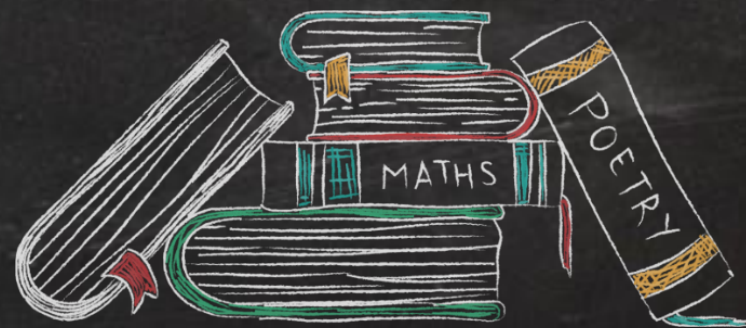


04 光照模型分析



$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

3 作业·示范·准备工作



01 准备1

1. 以FlatCol(L03)为模板, CtrlCV大法;
2. 修改路径名;
3. 追加Include文件:
 1. AutoLight.cginc;
 2. Lighting.cginc;
4. 考虑输入结构, 追加:
 1. uv0;
 2. normal;
 3. tangent;
5. 考虑输出结构, 追加:
 1. posWS;
 2. uv0;
 3. nDirWS;
 4. tDirWS;
 5. bDirWS;
 6. LIGHTING_COORDS(x, x);
6. 顶点Shader追加相应代码:
7. 以上可以OldSchoolPro作为参考, 复制粘贴完成;

```
Shader "AP01/L12/Dota2" {
    Properties {
    }
    SubShader {
        Tags {
            "RenderType"="Opaque"
        }
        Pass {
            Name "FORWARD"
            Tags {
                "LightMode"="ForwardBase"
            }
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag
            #include "UnityCG.cginc"
            #include "AutoLight.cginc"
            #include "Lighting.cginc"
            #pragma multi_compile_fwdbase_fullshadows
            #pragma target 3.0
            // 输入结构
            struct VertexInput {
                float4 vertex : POSITION;
                float2 uv : TEXCOORD0;
                float3 normal : NORMAL;
                float4 tangent : TANGENT;
            };
            // 输出结构
            struct VertexOutput {
                float4 pos : SV_POSITION;
                float3 posWS : TEXCOORD0;
                float2 uv : TEXCOORD1;
                float3 nDirWS : TEXCOORD2;
                float3 tDirWS : TEXCOORD3;
                float3 bDirWS : TEXCOORD4;
                LIGHTING_COORDS(5, 6)
            };
            // 输入结构>>>顶点Shader>>>输出结构
            VertexOutput vert (VertexInput v) {
                VertexOutput o = (VertexOutput)0;
                o.pos = UnityObjectToClipPos(v.vertex);
                o.posWS = mul(unity_ObjectToWorld, v.vertex).xyz;
                o.uv = v.uv;
                o.nDirWS = UnityObjectToWorldNormal(v.normal);
                o.tDirWS = normalize(mul(unity_ObjectToWorld, float4(v.tangent.xyz, 0.0)).xyz);
                o.bDirWS = normalize(cross(o.nDirWS, o.tDirWS) * v.tangent.w);
                TRANSFER_VERTEX_TO_FRAGMENT(o)
                return o;
            }
            // 输出结构>>>像素
            float4 frag(VertexOutput i) : COLOR {
                return float4(0.0, 1.0, 0.0, 1.0);
            }
            ENDCG
        }
    }
}
```


02 准备2

1. 把规划好的贴图都定义在材质面板上:

1. _MainTex:

1. RGB: Color,
2. A: Opacity;

2. _MaskTex:

1. R: SpecInt,
2. G: RimInt,
3. B: TintMask,
4. A: SpecPow;

3. _NormTex:

4. _MatelnessMask:

5. _EmissionMask

6. _DiffuseWarpTex

7. _FresnelWarpTex

8. _Cubemap

2. 对应声明输入参数:



```
Properties {  
    _MainTex      ("RGB:颜色 A:透贴", 2d) = "white"{}  
    _MaskTex      ("R:高光强度 G:边缘光强度 B:高光染色 A:高光次幂", 2d) = "black"{}  
    _NormTex      ("RGB:法线贴图", 2d) = "bump"{}  
    _MatelnessMask ("金属度遮罩", 2d) = "black"{}  
    _EmissionMask ("自发光遮罩", 2d) = "black"{}  
    _DiffWarpTex  ("颜色Warp图", 2d) = "gray"{}  
    _FresWarpTex  ("菲涅尔Warp图", 2d) = "gray"{}  
    _Cubemap      ("环境球", cube) = "_Skybox"{}  
}
```



```
...  
#pragma target 3.0  
// 输入参数  
uniform sampler2D _MainTex;  
uniform sampler2D _MaskTex;  
uniform sampler2D _NormTex;  
uniform sampler2D _MatelnessMask;  
uniform sampler2D _EmissionMask;  
uniform sampler2D _DiffWarpTex;  
uniform sampler2D _FresWarpTex;  
uniform samplerCUBE _Cubemap;  
// 输入结构  
...
```

03 准备3

1. 在像素Shader中准备好向量:

1. nDirWS,
2. vDirWS,
3. vrDirWS,
4. lDirWS,
5. lrDirWS;

2. 准备好中间量:

1. ndotl,
2. ndotv,
3. vdotlr;

```
// 输出结构>>>像素
float4 frag(VertexOutput i) : COLOR {
    // 向量准备
    {
        half3 nDirTS = UnpackNormal(tex2D(_NormTex, i.uv0));
        half3x3 TBN = half3x3(i.tDirWS, i.bDirWS, i.nDirWS);
        half3 nDirWS = normalize(mul(nDirTS, TBN));
        half3 vDirWS = normalize(_WorldSpaceCameraPos.xyz - i.posWS);
        half3 vrDirWS = reflect(-vDirWS, nDirWS);
        half3 lDirWS = _WorldSpaceLightPos0.xyz;
        half3 lrDirWS = reflect(-lDirWS, nDirWS);
    }
    // 中间量准备
    {
        half ndotl = dot(nDirWS, lDirWS);
        half ndotv = dot(nDirWS, vDirWS);
        half vdotlr = dot(vDirWS, lrDirWS);
    }
    // 返回值
    return float4(0.0, 1.0, 0.0, 1.0);
}
ENDCG
```

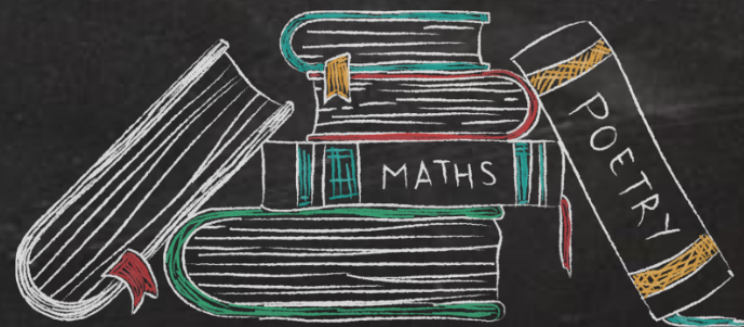

04 准备4

1. 纹理采样:
2. 信息提取:

```
① { // 采样纹理
    half4 var_MainTex = tex2D(_MainTex, i.uv);
    half4 var_MaskTex = tex2D(_MaskTex, i.uv);
    half var_MatelnessMask = tex2D(_MatelnessMask, i.uv).r;
    half var_EmissionMask = tex2D(_EmissionMask, i.uv).r;
    half3 var_FresWarpTex = tex2D(_FresWarpTex, ndotv).rgb;
    half3 var_Cubemap = texCUBElod(_Cubemap, float4(vrDirWS, lerp(8.0, 0.0, var_MaskTex.a))).rgb;
    // 提取信息
    ② {
        half3 baseCol = var_MainTex.rgb;
        half opacity = var_MainTex.a;
        half specInt = var_MaskTex.r;
        half rimInt = var_MaskTex.g;
        half specTint = var_MaskTex.b;
        half specPow = var_MaskTex.a;
        half matellic = var_MatelnessMask;
        half emitInt = var_EmissionMask;
        half3 envCube = var_Cubemap;
        half shadow = LIGHT_ATTENUATION(i);
    }
}
```

$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

3 作业·示范·光照模型



01 DiffCol SpecCol

// 光照模型

// 漫反射颜色 镜面反射颜色

```
half3 diffCol = lerp(baseCol, half3(0.0, 0.0, 0.0), matellic);
```

```
half3 specCol = lerp(baseCol, half3(0.3, 0.3, 0.3), specTint) * specInt;
```

从Color纹理, Metalness, TintByBase两个Mask, 计算出DiffCol, SpecCol:

- $\text{half3 diffCol} = \text{lerp}(\text{baseCol}, \text{half3}(0.0, 0.0, 0.0), \text{matellic});$
- $\text{half3 specCol} = \text{lerp}(\text{baseCol}, \text{half3}(0.3, 0.3, 0.3), \text{specTint}) * \text{specInt};$
 - 其中0.3为经验值;
- DiffCol可理解为Diffuse贴图, SpecCol可理解为SpecularColor贴图;



02 Fresnel

// 菲涅尔

```
half3 fresnel = lerp(var_FresWarpTex, 0.0, metallic);  
half fresnelCol = fresnel.r;    // 无实际用途  
half fresnelRim = fresnel.g;  
half fresnelSpec = fresnel.b;
```

1. 考虑金属，非金属的菲涅尔特性，对Fresnel值做Lerp操作；削弱金属部分Fresnel；
2. 提取对各通道Fresnel值：
 - R: FresnelCol: 应是两种DiffWarp的插值Mask，官方申明罕见使用，在此不考虑；
 - G: FresnelRim: 轮廓光用Fresnel；
 - B: FresnelSpec: 镜面反射用Fresnel；



03 DirDiff·主光漫反射

1. 材质面板追加主光颜色: `_LightCol`;
2. 对应声明输入参数;
3. 主光漫反射采用RampTex:
 1. 计算HalfLambert;
 2. 构造RampUV;
 3. 采样RampTex;
 4. 混合DiffCol, Ramp结果, 和主光颜色;

```
① _LightCol      ("光颜色", color) = (1.0, 1.0, 1.0, 1.0)

② uniform half3 _LightCol;

③ // 光源漫反射
half halfLambert = ndot1 * 0.5 + 0.5;
half3 var_DiffWarpTex = tex2D(_DiffWarpTex, half2(halfLambert, 0.2));
half3 dirDiff = diffCol * var_DiffWarpTex * _LightCol;
```



04 DirSpec · 主光镜面反射

1. 材质面板追加:

1. _SpecPow: 高光次幂,
2. _SpecInt: 高光强度;

2. 对应声明输入参数;

3. 主光镜面反射采用Phong+FresnelSpec:

1. 计算Phong;
2. Phong乘以Lambert;
3. Phong和FresnelSpec混合;
4. 再混合SpecCol, 和主光颜色;

① _SpecPow ("高光次幂", range(0.0, 30.0)) = 5
_SpecInt ("高光强度", range(0.0, 10.0)) = 5

② uniform half _SpecPow;
uniform half _SpecInt;

③ // 光源镜面反射
half phong = pow(max(0.0, vdotr), specPow * _SpecPow);
half spec = phong * max(0.0, ndotl);
spec = max(spec, fresnelSpec);
spec = spec * _SpecInt;
half3 dirSpec = specCol * spec * _LightCol;



05 EnvDiff·环境漫反射

1. 材质面板追加:

1. _EnvCol: 环境光颜色,
2. _EnvDiffInt: 环境漫反射强度;

2. 对应声明输入参数;

3. 缓解漫反射采用单色环境光:

1. 直接混合DiffCol, EnvCol, EnvDiffInt即可

① _EnvCol ("环境光颜色", color) = (1.0, 1.0, 1.0, 1.0)
_EnvDiffInt ("环境漫反射强度", range(0.0, 5.0)) = 0.5

② uniform half3 _EnvCol;
uniform half _EnvDiffInt;

③ // 环境漫反射
half3 envDiff = diffCol * _EnvCol * _EnvDiffInt;



06 EnvSpec • 环境镜面反射

1. 材质面板追加:

1. _EnvSpecInt: 环境镜面反射强度;

2. 对应声明输入参数;

3. 环境镜面反射采用Cubemap:

1. 计算ReflectInt反射率;

2. 混合SpecCol, 反射率, 反射环境, 反射强度;

①

```
_EnvSpecInt ("环境镜面反射强度", range(0.0, 10.0)) = 0.5
```

②

```
uniform half _EnvSpecInt;
```

③

```
// 环境镜面反射  
half reflectInt = max(fresnelSpec, matellic) * specInt;  
half3 envSpec = specCol * reflectInt * envCube * _EnvSpecInt;
```



07 RimLight·轮廓光

1. 材质面板追加:

1. `_RimCol`: 轮廓光颜色,

2. `_RimInt`: 轮廓光强度;

2. 对应声明输入参数;

3. 缓解漫反射采用单色环境光:

1. 混合`RimCol`, `RimCol`, `FresnelRim`, 法线
G通道, `RimInt`;

①

```
_RimCol      ("轮廓光颜色", color) = (1.0, 1.0, 1.0, 1.0)  
_RimInt      ("轮廓光强度", range(0.0, 3.0)) = 1.0
```

②

```
uniform half3 _RimCol;  
uniform half _RimInt;
```

③

```
// 轮廓光  
half3 rimLight = _RimCol * fresnelRim * rimInt * max(0.0, nDirWS.g) * _RimInt;
```



08 Emission·自发光

1. 材质面板追加:

1. `_EmitInt`: 自发光强度;

2. 对应声明输入参数;

3. 缓解漫反射采用单色环境光:

1. 混合`DiffCol`, `EmitInt`;

① `_EmitInt` ("自发光强度", `range(0.0, 10.0)`) = 1.0

② `uniform half _EmitInt;`

③ `// 自发光`
`half3 emission = diffCol * emitInt * _EmitInt;`



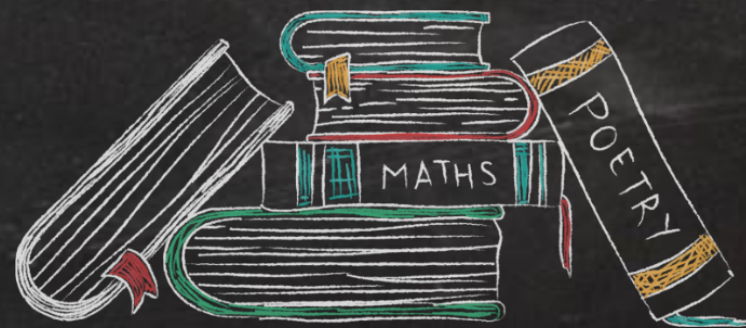
08 最终混合

最终混合



$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

4 作业·示范·其他细节



01 透明剪切

1. 材质面板追加:

1. `_Cutoff`: 透明剪切阈值;

2. 对应声明输入参数;

3. `Clip` (不透明度-透明剪切阈值);

```
① [HideInInspector]
   _Cutoff      ("Alpha cutoff", Range(0,1)) = 0.5

② uniform half _Cutoff;

③ // 透明剪切
   clip(opacity - _Cutoff);
```



02 透明剪切投影修正

1. 材质面板追加Fallback必须参数:

1. _Color: 颜色;

2. 对应声明输入参数;

3. 声明投影正确的回退Shader;

①

```
[HideInInspector]
```

```
_Color ("Main Color", Color) = (1,1,1,1)
```

②

```
uniform half3 _Color;
```

③

```
// 声明回退Shader
```

```
FallBack "Legacy Shaders/Transparent/Cutout/VertexLit"
```



03 开启双面显示

- PassTags后声明Cull模式-Cull Off;

```
...  
}  
Cull Off  
CGPROGRAM  
...
```



04 效果完成



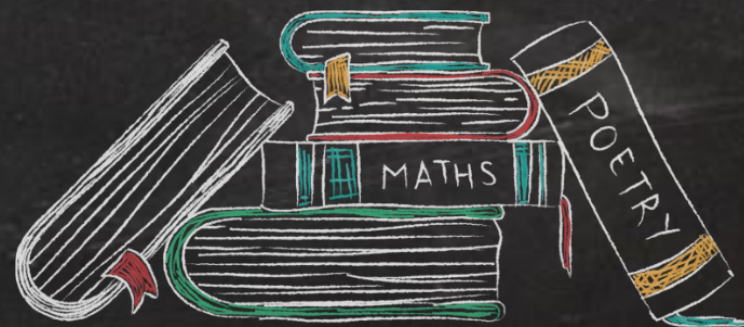
• 到此为止，仅是效果完成，正式项目中还有后续工作要做：

1. 对可选效果拆分Shader分支；
2. 整理优化材质面板；
3. 优化代码，验证兼容性；
4. 等...

$$\begin{array}{r} 24 \times 5 \\ \hline 33 \end{array}$$

5

情报·开源Shader



01 SD, SP

Substance Painter 2019 > resources > shelf > allegorithmic > shaders				
名称	修改日期	类型	大小	
.alg_meta	2019/12/19 7:31	文件夹		
custom-ui	2019/2/27 19:24	文件夹		
mdl	2019/2/27 19:24	文件夹		
adobe-dimension.glsl	2019/2/27 19:23	GLSL 文件	3 KB	
clay.glsl	2019/2/27 19:24	GLSL 文件	2 KB	
dota-2.glsl	2019/2/27 19:23	GLSL 文件	8 KB	
lens-studio.glsl	2019/2/27 19:24	GLSL 文件	23 KB	
non-pbr-spec-gloss.glsl	2019/2/27 19:23	GLSL 文件	3 KB	
pbr-car-paint.glsl	2019/2/27 19:23	GLSL 文件	3 KB	
pbr-coated.glsl	2019/2/27 19:23	GLSL 文件	5 KB	
pbr-material-layering.glsl	2019/2/27 19:23	GLSL 文件	13 KB	
pbr-material-layering-10-mats.glsl	2019/2/27 19:23	GLSL 文件	23 KB	
pbr-metal-rough.glsl	2019/2/27 19:24	GLSL 文件	3 KB	
pbr-metal-rough-anisotropy-angle.glsl	2019/2/27 19:23	GLSL 文件	3 KB	
pbr-metal-rough-with-alpha-blendin...	2019/2/27 19:23	GLSL 文件	3 KB	
pbr-metal-rough-with-alpha-test.glsl	2019/2/27 19:23	GLSL 文件	3 KB	
pbr-spec-gloss.glsl	2019/2/27 19:23	GLSL 文件	2 KB	
pbr-velvet.glsl	2019/2/27 19:23	GLSL 文件	3 KB	

Substance Designer 2019 > resources > view3d > shaders				
名称	修改日期	类型	大小	
blinn	2019/12/21 8:03	文件夹		
common	2019/11/14 15:13	文件夹		
lambrert	2019/12/21 8:03	文件夹		
mesh_info	2019/2/27 19:29	文件夹		
physically_metallic_roughness	2019/12/21 8:03	文件夹		
physically_metallic_roughness_coated	2019/11/14 15:13	文件夹		
physically_metallic_roughness_sss	2019/12/21 8:03	文件夹		
physically_specular_glossiness	2019/12/21 8:03	文件夹		
blinn.glslfx	2019/12/21 8:03	GLSLFX 文件	7 KB	
glslfx_file_format.txt	2019/11/14 15:12	文本文档	23 KB	
lambrert.glslfx	2019/12/21 8:03	GLSLFX 文件	3 KB	
mesh_info.glslfx	2019/2/27 19:29	GLSLFX 文件	3 KB	
physically_metallic_roughness.glslfx	2019/12/21 8:03	GLSLFX 文件	7 KB	
physically_metallic_roughness_coated...	2019/12/21 8:03	GLSLFX 文件	13 KB	
physically_metallic_roughness_sss.glslfx	2019/12/21 8:03	GLSLFX 文件	11 KB	
physically_specular_glossiness.glslfx	2019/12/21 8:03	GLSLFX 文件	7 KB	

- SP, SD都有GLSL的Shader源码; Dota2, PBR, 都有;

02 8猴

Marmoset > Toolbag 3 > data > shader > mat				
名称	修改日期	类型	大小	
albedo	2020/3/16 1:04	文件夹		
custom	2020/3/16 1:04	文件夹		
diffusion	2020/3/16 1:04	文件夹		
displacement	2020/3/16 1:04	文件夹		
emissive	2020/3/16 1:04	文件夹		
merge	2020/3/16 1:04	文件夹		
microsurface	2020/3/16 1:04	文件夹		
occlusion	2020/3/16 1:04	文件夹		
other	2020/3/16 1:04	文件夹		
reflection	2020/3/16 1:04	文件夹		
reflectivity	2020/3/16 1:04	文件夹		
subdivision	2020/3/16 1:04	文件夹		
surface	2020/3/16 1:04	文件夹		
transparency	2020/3/16 1:04	文件夹		
light.frag	2018/2/10 4:50	FRAG 文件	3 KB	
mat.dom	2016/10/25 6:25	DOM 文件	3 KB	
mat.frag	2019/2/1 7:17	FRAG 文件	4 KB	
mat.geom	2017/6/30 8:27	GEOM 文件	2 KB	
mat.hull	2016/10/25 6:25	HULL 文件	3 KB	
mat.vert	2017/6/30 8:27	VERT 文件	2 KB	
state.dom	2016/4/13 5:53	DOM 文件	1 KB	
state.frag	2019/2/1 7:17	FRAG 文件	2 KB	
...	2016/10/25 6:25	GEOM 文件	1 KB	

- 8猴的Shader也一样，都是有源码的；
- 8猴的结构稍微难懂一点；



Thanks